

American International University-Bangladesh
Department of Computer Science

CSC4164

Advanced Programming

.NET / ASP.NET Core MVC

Lab 3

Model Binding & Validation

Course Code	CSC4164
Course Title	Advanced Programming (.NET / ASP.NET Core MVC)
Lab Session	Lab 3 — Week 3
Instructor	Jahid Hassan
Duration	2 Hours 20 Minutes (140 minutes)
Work Mode	Individual
Lab Task	None this session
Prerequisite	Lab 2 completed — routing and views working

1. Lab Objectives

By the end of this lab session, you will be able to:

- Understand how ASP.NET Core MVC binds HTTP request data to action method parameters
- Bind form data to a simple parameter and to a complex model object
- Apply built-in data annotation attributes to validate model properties
- Display validation error messages in a Razor view using tag helpers
- Create a custom validation attribute for business-specific rules

2. Prerequisites

- Lab 2 completed — routing and views working in FirstMVCApp
- Theory class Week 3 attended — Module 4 (Model Binding) and Module 5 (Custom Validation)

3. Theory Recap

3.1 Model Binding

Model binding is the process by which ASP.NET Core automatically maps data from an HTTP request to action method parameters. It reads from three sources in order:

Source	Description	Example
Form data	Values submitted via an HTML form POST	Name field in a registration form
Route data	Values embedded in the URL path	/Student/Details/5 → id = 5
Query string	Values appended to the URL after ?	/search?keyword=mvc → keyword = mvc

3.2 Data Annotation Attributes

Data annotations are attributes applied to model properties to define validation rules. ASP.NET Core checks these automatically when model binding occurs.

Attribute	Purpose	Example
[Required]	Field must not be empty	[Required] public string Name { get; set; }
[StringLength(max)]	Limits string length	[StringLength(50)] public string Name { get; set; }
[Range(min, max)]	Numeric value must be within range	[Range(0, 100)] public int Marks { get; set; }
[EmailAddress]	Must be a valid email format	[EmailAddress] public string Email { get; set; }
[RegularExpression]	Must match a regex pattern	[RegularExpression(@"^\d{4}\$")] public string Year

Attribute	Purpose	Example
[Compare("Property")]	Two fields must match	[Compare("Password")] public string Confirm

3.3 ModelState and Validation Flow

When a form is submitted, ASP.NET Core runs validation and stores the result in ModelState. The controller checks ModelState.IsValid before proceeding:

```
[HttpPost]
public IActionResult Register(Student student)
{
    if (!ModelState.IsValid)
    {
        // Validation failed – return the form with error messages
        return View(student);
    }

    // Validation passed – process the data
    return RedirectToAction("Index");
}
```

4. Lab Activities

Warning: Important: Type all code manually. Do not copy and paste from this manual. Typing the code builds muscle memory, helps you spot errors, and improves your understanding of the syntax.

Task 1 Create the Student Model (15 min)

You will create a Student model class with properties and data annotation attributes.

Step 1 — Create the model class

1. Right-click the Models folder → Add → Class
2. Name it Student.cs and click Add
3. Type the following code:

```
using System.ComponentModel.DataAnnotations;

namespace FirstMVCApp.Models
{
    public class Student
    {
        [Required(ErrorMessage = "Student ID is required")]
        [StringLength(10, ErrorMessage = "ID cannot exceed 10 characters")]
        public string StudentId { get; set; }

        [Required(ErrorMessage = "Name is required")]
        [StringLength(100, MinimumLength = 3, ErrorMessage = "Name must be between 3 and 100 characters")]
        public string Name { get; set; }

        [Required(ErrorMessage = "Email is required")]
        [EmailAddress(ErrorMessage = "Please enter a valid email address")]
        public string Email { get; set; }

        [Required(ErrorMessage = "Age is required")]
        [Range(16, 40, ErrorMessage = "Age must be between 16 and 40")]
        public int Age { get; set; }

        [Required(ErrorMessage = "GPA is required")]
        [Range(0.0, 4.0, ErrorMessage = "GPA must be between 0.0 and 4.0")]
        public double GPA { get; set; }
    }
}
```

Task 2 Create the Registration Form (25 min)

You will create a StudentController with GET and POST actions, and build the registration form view and add this controller to your existing menu under “_Layout.cshtml” page.

Step 1 — Add RegistrationController

4. Right-click Controllers → Add → Controller → MVC Controller — Empty
5. Name it RegistrationController and type the following:

```
using FirstMVCApp.Models;
using Microsoft.AspNetCore.Mvc;

namespace FirstMVCApp.Controllers
{
    public class RegistrationController : Controller
    {
        // GET: Show the empty registration form
        [HttpGet]
        public IActionResult Register()
        {
            return View();
        }

        // POST: Receive and validate the submitted form
        [HttpPost]
        public IActionResult Register(Student student)
        {
            if (!ModelState.IsValid)
            {
                return View(student);
            }

            // Validation passed — show success
            ViewBag.Message = "Registration successful for " + student.Name;
            return View("Success", student);
        }
    }
}
```

Step 2 — Create the Register view

6. Create Views/Registration/ folder
7. Add Register.cshtml and type the following:

```
@model FirstMVCApp.Models.Student

@{
    ViewData["Title"] = "Student Registration";
}

<h2>Student Registration</h2>

<form action="" method="post">

    @Html.ValidationSummary()

    <div class="mb-3">
        <label asp-for="StudentId"></label>
        <input type="text" asp-for="StudentId" class="form-control" />
        <span asp-validation-for="StudentId" class="text-danger"></span>
    </div>
```

```

<div class="mb-3">
    <label asp-for="Name"></label>
    <input type="text" asp-for="Name" class="form-control" />
    <span asp-validation-for="Name" class="text-danger"></span>
</div>

<div class="mb-3">
    <label asp-for="Email"></label>
    <input type="text" asp-for="Email" class="form-control" />
    <span asp-validation-for="Email" class="text-danger"></span>
</div>

<div class="mb-3">
    <label asp-for="Age"></label>
    <input type="number" asp-for="Age" class="form-control"/>
    <span asp-validation-for="Age" class="text-danger"></span>
</div>

<div class="mb-3">
    <label asp-for="GPA"></label>
    <input type="text" asp-for="GPA" class="form-control" />
    <span asp-validation-for="GPA" class="text-danger"></span>
</div>

<div>
    <button class="btn btn-info" type="submit">Register</button>
</div>

</form>

```

Step 3 — Create the Success view

8. Add Success.cshhtml in Views/Registration/ and type the following:

```

@model FirstMVCApp.Models.Student

@{
    ViewData["Title"] = "Registration Successful";
}

<h2>Registration Successful</h2>

<table class="table table-bordered">
    <tr>
        <th>Field</th>
        <th>Value</th>
    </tr>
    <tr>
        <td>Student ID</td>
        <td>@Model.StudentId</td>
    </tr>
    <tr>
        <td>Name</td>
        <td>@Model.Name</td>
    </tr>
    <tr>
        <td>Email</td>

```

```
        <td>@Model.Email</td>
    </tr>
    <tr>
        <td>Age</td>
        <td>@Model.Age</td>
    </tr>
    <tr>
        <td>GPA</td>
        <td>@Model.GPA</td>
    </tr>
</table>

<br />
<a asp-controller="Registration" asp-action="Register">Register Another
Student</a>
```

Step 4 — Test server-side validation

9. Run the application and navigate to /Registration/Register
10. Click Register without filling in any fields
11. Confirm all validation error messages appear next to each field
12. Enter an invalid email (e.g. notanemail) and confirm the email error shows
13. Enter Age = 50 and confirm the range error shows
14. Fill all fields correctly and confirm the Success view displays the submitted data

Task 3 Display a Validation Summary (15 min)

The validation summary is already included in your form via `@Html.ValidationSummary()`. In this task you will observe how it behaves and understand the difference between field-level and summary-level errors.

Step 1 — Observe the validation summary

Your Register.cshtml already includes `@Html.ValidationSummary()` at the top of the form. Run the app and observe its behaviour:

15. Run the application and navigate to /Registration/Register
16. Submit an empty form — confirm all errors appear in the summary and next to each field
17. Update the call to pass true to show only model-level errors:

```
// Show only model-level errors, not individual field errors
@Html.ValidationSummary(true)
```

18. Submit again and observe the summary no longer repeats individual field errors
19. Revert back to `@Html.ValidationSummary()` for full summary

Note: `@Html.ValidationSummary()` with no argument shows all errors. Passing true shows only model-level errors not tied to a specific field.

Task 4 Create a Custom Validation Attribute (30 min)

Built-in attributes cover common cases but cannot handle business-specific rules. You will create a custom attribute that ensures a date of birth is in the past and the student is at least 16 years old.

Step 1 — Create a Validations folder

20. Right-click the project root in Solution Explorer → Add → New Folder
21. Name it Validations

Step 2 — Create the custom attribute class

22. Right-click the Validations folder → Add → Class
23. Name it MinimumAgeAttribute.cs and type the following:

```
using System.ComponentModel.DataAnnotations;

namespace FirstMVCApp.Validations
{
    public class MinimumAgeAttribute : ValidationAttribute
    {
        private readonly int _minimumAge;

        public MinimumAgeAttribute(int minimumAge)
        {
            _minimumAge = minimumAge;
            ErrorMessage = $"Student must be at least {minimumAge} years old";
        }

        protected override ValidationResult IsValid(
            object value, ValidationContext validationContext)
        {
            if (value is DateTime dateOfBirth)
            {
                // Date must be in the past
                if (dateOfBirth >= DateTime.Today)
                {
                    return new ValidationResult(
                        "Date of birth must be in the past");
                }

                // Calculate age
                int age = DateTime.Today.Year - dateOfBirth.Year;
                if (dateOfBirth > DateTime.Today.AddYears(-age)) age--;

                if (age < _minimumAge)
                {
                    return new ValidationResult(ErrorMessage);
                }
            }

            return ValidationResult.Success;
        }
    }
}
```

Step 3 — Add DateOfBirth to the Student model

Open Models/Student.cs and add the following property after the GPA property and comment the age property:

```
using FirstMVCAApp.Validations;

// Add this property to the Student class
[Required(ErrorMessage = "Date of birth is required")]
[MinimumAge(16)]
public DateTime DateOfBirth { get; set; }
```

Step 4 — Replace the age field with DateOfBirth field in Register View

```
<div class="mb-3">
  <label asp-for="DateOfBirth"></label>
  <input type="date" asp-for="DateOfBirth" class="form-control"/>
  <span asp-validation-for="DateOfBirth" class="text-danger"></span>
</div>
```

Step 5 — Also replace it to the Success view

Open Success.cshtml and add a new row to the table:

```
<tr>
  <td>Date of Birth</td>
  <td>@Model.DateOfBirth.ToString("dd MMM yyyy")</td>
</tr>
```

Step 6 — Test the custom validation

24. Run the application and navigate to /Registration/Register
25. Enter a future date as Date of Birth — confirm the error: Date of birth must be in the past
26. Enter a date that makes the student younger than 16 — confirm the age error
27. Enter a valid date for a 20-year-old student — confirm the form submits successfully

Task 5 Explore ModelState Errors Manually (10 min)

You will manually add a custom error to ModelState to understand how it works independently of data annotations.

Step 1 — Add a manual ModelState error

In RegistrationController, update the POST action to add a manual check after the annotation validation:

```
[HttpPost]
public IActionResult Register(Student student)
{
```

```
// Check for duplicate student ID manually
if (student.StudentId == "STU001")
{
    ModelState.AddModelError("StudentId",
        "Student ID STU001 is already registered");
}

if (!ModelState.IsValid)
{
    return View(student);
}

ViewBag.Message = "Registration successful for " + student.Name;
return View("Success", student);
}
```

28. Run the application and submit the form with StudentId = STU001
29. Confirm the manual error message appears next to the Student ID field
30. Submit with a different ID and confirm the form proceeds normally

Note: `ModelState.AddModelError` is how you add errors from database checks, business rules, or any logic that cannot be expressed as a simple attribute.

5. Common Errors and Solutions

Error	Likely Cause	Solution
Validation errors do not appear in view	@Html.ValidationMessage() property name does not match model	Ensure the string passed to @Html.ValidationMessage() exactly matches the model property name
Client-side validation not working	jQuery validation scripts not loaded in layout	Confirm all three jQuery validation scripts are in _Layout.cshtml
ModelState.IsValid is always false	Required field has no default and is not submitted	Check that all [Required] fields have a corresponding input in the form
Custom attribute not firing	using statement missing or namespace incorrect	Add using FirstMVCApp.Validations to the model file
Date field shows wrong format on Success page	Default DateTime.ToString() format is verbose	Use @Model.DateOfBirth.ToString("dd MMM yyyy") for clean output

6. Completion Checklist

#	Checklist Item	Done
1	Student model created with StudentId, Name, Email, Age, GPA, DateOfBirth properties	☐
2	All properties annotated with appropriate data annotation attributes	☐
3	RegistrationController created with GET and POST Register actions	☐
4	Register.cshtml form created with plain HTML inputs, labels with for= attribute, and @Html.ValidationMessage()	☐
5	Success.cshtml created and displays all submitted model properties	☐
6	Server-side validation tested — errors appear on empty or invalid submission	☐
8	@Html.ValidationSummary() observed and tested with true and false arguments	☐
9	MinimumAgeAttribute custom class created in Validations folder	☐
10	Custom attribute applied to DateOfBirth and tested with past and future dates	☐
11	Manual ModelState.AddModelError tested with duplicate StudentId	☐

7. Self-Assessment Questions

Review these questions before leaving. They may appear in Quiz 1.

- What are the three sources from which model binding reads data?
- What does ModelState.IsValid return when a [Required] field is left empty?
- What base class must a custom validation attribute inherit from?

- What does `ModelState.AddModelError` do and when would you use it?
- What is the difference between `@Html.ValidationSummary()` and `@Html.ValidationSummary(true)`?

8. Further Reading

- Model binding in ASP.NET Core: <https://learn.microsoft.com/aspnet/core/mvc/models/model-binding>
- Model validation in ASP.NET Core: <https://learn.microsoft.com/aspnet/core/mvc/models/validation>
- Custom validation attributes: <https://learn.microsoft.com/aspnet/core/mvc/models/validation#custom-attributes>

End of Lab 3 — Lab 4 covers Database-First EF Core.